BCT 2408 COMPUTER ARCHITECTURE

LAB 5

EARL KINUTHIA SCT 212-0067/2019

A:

Cache size = 16 KB = 16 × 1024 = 16,384 byte

Associativity = Direct-mapped

Cache line size = 64 bytes Cache uses physical addressing

Array element size = 4 bytes (single precision float)

Number of elements in X and Y = 4096 each

X and Y are allocated consecutively in physical memory

The loop runs 4096 iterations, and in each iteration:

- 1 element from X is loaded
- 1 element from Y is loaded
- 1 element of X is written back

This means 3 memory operations per iteration:

- Load X[i]
- Load Y[i]
- Store X[i]
- X occupies 4096 × 4 = 16 KB
- Y also occupies 16 KB
- Together, they occupy 32 KB, twice the size of the data cache, and they're allocated consecutively

Cache Mapping Details

- Line size = 64 bytes, so number of cache lines = 16 KB / 64 = 256 lines
- Each cache line can hold 64 / 4 = 16 elements (since each element is 4 bytes)

Therefore:

- X spans 4096 / 16 = 256 cache lines
- Y also spans 256 cache lines

Because it's a direct-mapped cache, and X and Y are consecutive in memory, their addresses map to the same cache lines:

Line 0 in X maps to line 0 in cache

Line 0 in Y also maps to line 0 in cache → conflict!

Compulsory Misses:

These happen the first time a block is accessed.

For X, each 64-byte block (16 elements) will be loaded once:

256 blocks → 256 compulsory misses

For Y, same → 256 compulsory misses

So, total compulsory misses = 256 + 256 = 512

Conflict Misses:

Due to direct-mapped nature, X and Y compete for the same cache lines.

- After X[0] is loaded into cache line 0, Y[0] replaces it.
- Then storing back to X[0] causes another load (miss) since it was replaced → this keeps happening

Every access to X and Y will cause a miss after the first one because of this conflict.

So for each of the 4096 iterations:

X[i] load: always a miss (conflict after first access)

Y[i] load: always a miss (conflict)

X[i] store: always a miss (since X[i] was just evicted by Y[i])

Thus:

4096 X loads → 4096 misses

4096 Y loads → 4096 misses

4096 X stores → 4096 misses

From these, subtract the 512 compulsory misses, so:

Conflict misses = 12288 - 512 = 11776

There are no capacity misses, because 32 KB total accessed data exceeds cache size, but the individual working set at any moment is small (just 2 floats at a time).

Hence Miss Rate:

Total data memory accesses = 3 × 4096 = 12288

Miss rate = 12288 / 12288 = 1.0 or 100%

B:

Direct-mapped cache = 256 lines

Arrays X and Y map to the same set of cache lines → conflict misses

Cause: X and Y are laid out back-to-back in memory

The idea is to prevent the conflict by offsetting Y in memory so it does not conflict with X. Insert a padding array between X and Y in memory. For example, allocate Y with an offset so that it maps to different cache lines than X.

Summary:

- X occupies cache lines 0–255
- Pad occupies lines 256–511 (will not be accessed)
- Y starts from line 256, wrapping back to 0 (but doesn't conflict with X anymore if spaced correctly)

Since the cache only has 256 lines, offsetting Y by at least one cache size (16KB) ensures X and Y don't overlap in cache. Now X and Y map to distinct sets of cache lines → no more conflict misses.

Each array still spans 256 cache blocks (64B lines), so:

Compulsory Misses:

- 256 lines for X → 256 compulsory misses (on first load)
- 256 lines for Y → 256 compulsory misses
- Total Compulsory = 512

Conflict Misses:

- None, because X and Y now map to different cache sets

Capacity Misses:

- Still none — only two floats are used at any time (working set small)

Total Cache Misses = 512

Miss Rate (Post Optimization)**:**

- Total accesses = 3 × 4096 = 12288
- Miss rate = 512 / 12288 ≈ 0.0417 = 4.17%

C:

Hardware Optimization: Use a 2-Way Set-Associative Cache

Current Cache (Baseline)

- Direct mapped: 1 line per set → high chance of conflict
- 16KB total, 64B per line = 256 cache lines
- 256 sets, 1-way

Change: Make Cache 2-Way Set Associative

- Same cache size: 16 KB
- Line size: 64 bytes
- Now: 2-way set-associative → each set has 2 lines
- Number of sets = (16KB / 64B) / 2 = 128 sets

Effect of the Change

- With 2-way associativity, two blocks mapping to the same set can now coexist
- So X[i] and Y[i] no longer conflict as often as in the direct-mapped version
- Unless more than 2 blocks map to the same set at the same time (which they don't), conflict misses are eliminated

Compulsory Misses:

- Each new 64-byte block fetched causes a compulsory miss on its first access
- X spans 4096 elements / 16 elements per block = 256 blocks
- Y also spans 256 blocks
- So: 256 (X) + 256 (Y) = 512 compulsory misses

Conflict Misses:

- None: Since it's 2-way set-associative, X[i] and Y[i] can coexist in the same set

Capacity Misses:

- Still none: Working set (just a few floats at a time) is well below 16 KB

- Total Cache Misses = 512

Miss Rate:

- Total memory accesses = 3 × 4096 = 12288
- Miss rate = 512 / 12288 ≈ 4.17%